08

Định dạng file dữ liệu

Parquet · Avro · ORC · Delta — hiểu để chọn đúng

Hôm nay mình nói gì

1. Row vs column layout
2. Nén và encoding
3. Schema evolution
4. Predicate pushdown
5. Partitioning & small-file problem

Row layout vs Column layout trên đĩa

Row · CSV, Avro

r1c1	r1c2	r1c3	r1c4
r2c1	r2c2	r2c3	r2c4
r3c1	r3c2	r3c3	r3c4
r4c1	r4c2	r4c3	r4c4

SELECT c3 → đọc 16 ô, dùng 4

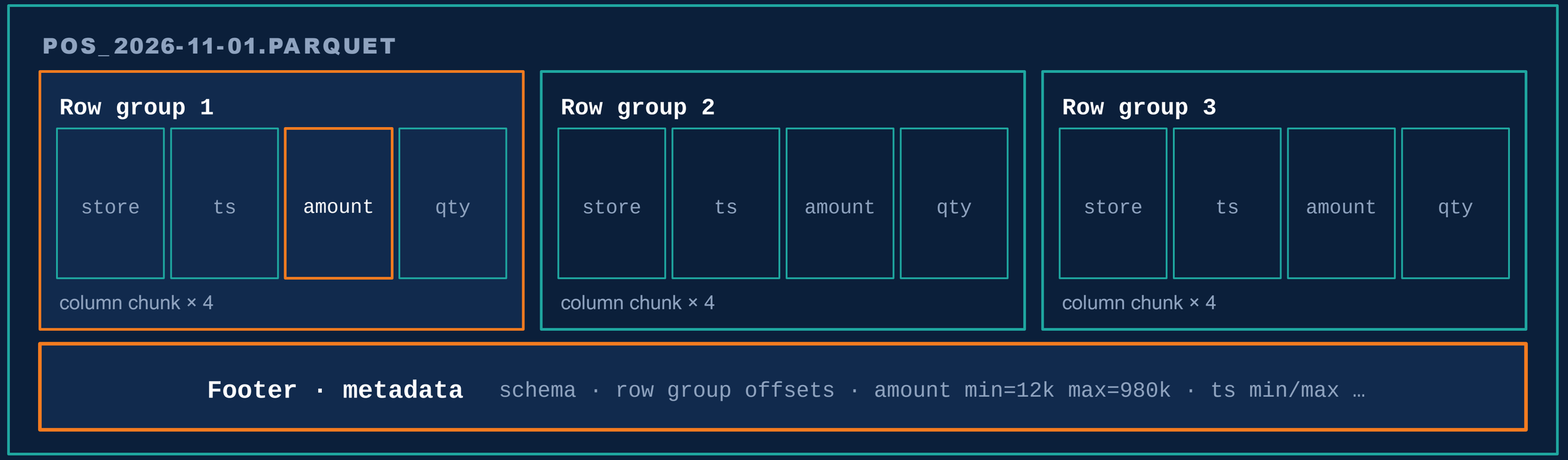
Column · Parquet, ORC

r1c1	r2c1	r3c1	r4c1
r1c2	r2c2	r3c2	r4c2
r1c3	r2c3	r3c3	r4c3
r1c4	r2c4	r3c4	r4c4

→ **SELECT c3 → đọc đúng 1 đoạn liền**

Phân tích đọc ít cột, nhiều dòng → column.

Bên trong file Parquet



Min/max trong footer là cách Parquet bỏ qua dữ liệu.

Nén và encoding

- Dictionary encoding cho cột ít giá trị (store_id)
- Run-length cho cột lặp
- Snappy nhanh, Zstd nhỏ hơn
- Nén theo cột hiệu quả hơn nén theo dòng

Parquet vs Avro vs ORC vs Delta / Iceberg

	Parquet	Avro	ORC	Delta / Iceberg
Layout	Column	Row	Column	Column + metadata
Mạnh về	Phân tích	Streaming, schema evolution	Hive ecosystem	ACID, time travel
Schema evolution	Thêm cột; đổi kiểu khó	Tốt, schema đi kèm file	Thêm cột	Đầy đủ, có version
Trong dự án	Lake silver / gold	Kafka message	Ít gặp	Lakehouse, từ video 17

Predicate pushdown

- WHERE đẩy xuống tầng đọc file
- Dùng min/max để bỏ row group
- Chỉ đọc cột trong SELECT
- Hiệu quả khi dữ liệu được sort theo cột lọc

Partitioning trên lake

```
pos/
├── dt=2026-10-31/
│   ├── store=HN01/ part-0.parquet
│   └── store=SG01/ part-0.parquet
├── dt=2026-11-01/
│   ├── store=HN01/ part-0.parquet
│   └── store=SG01/ part-0.parquet
└── dt=2026-11-02/
    ├── store=HN01/ part-0.parquet
    └── store=SG01/ part-0.parquet
```

```
SELECT SUM(amount)
FROM pos
WHERE dt = '2026-11-01'
```

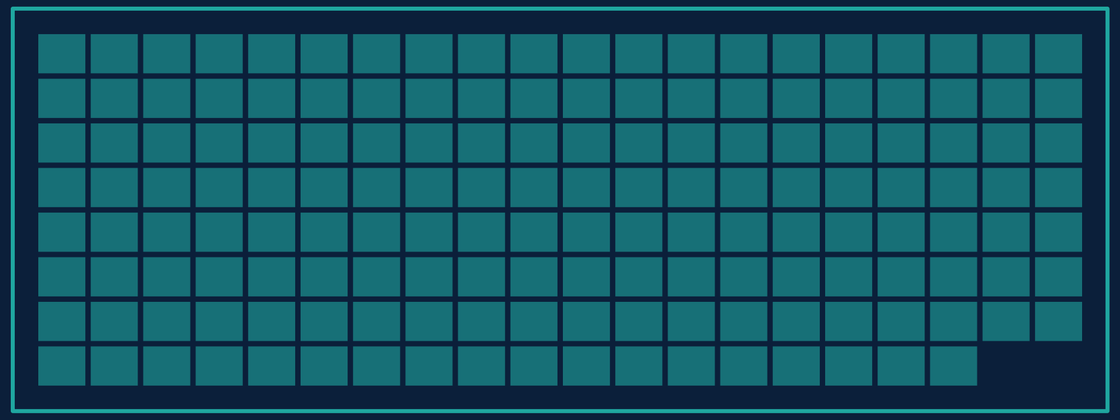
→ **chỉ chạm 1 nhánh**

2 nhánh còn lại không mở file nào

Partition theo cột hay lọc, không theo cột có hàng triệu giá trị.

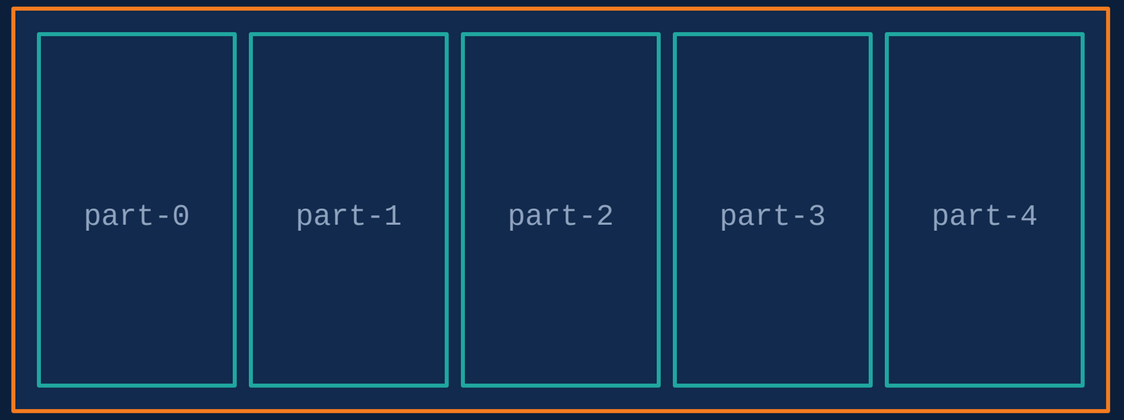
Small-file problem

5.000 file × 10 KB



5.000 lần mở file, 5.000 footer phải đọc

5 file × 10 MB



cùng 50 MB dữ liệu, 5 lần mở file

Mục tiêu 128 MB – 1 GB mỗi file.

Ghi Parquet đúng cách

```
import polars as pl

df = pl.scan_parquet("bronze/pos/*.parquet").collect()

df.sort("store_id", "ts").write_parquet(
    "silver/pos/",
    compression="zstd",
    row_group_size=1_000_000,
    partition_by=["dt"],
    statistics=True,
)
```

Zstd cho lưu lâu, Snappy cho dữ liệu đọc nóng. Sort trước để min/max hữu ích.

Lỗi hay gặp

SAI	ĐÚNG
~~Partition theo customer_id~~	Partition theo ngày, cluster theo store
~~Ghi file mỗi phút rồi để đó~~	Compaction định kỳ
~~CSV cho tầng silver~~	Parquet từ bronze trở đi
~~Lưu JSON thô làm nguồn phân tích~~	Flatten sang Parquet

Lab #08 — Benchmark 4 định dạng

1. Lấy 10 triệu dòng POS
2. Ghi ra CSV, JSON, Avro, Parquet (snappy, zstd)
3. Đo dung lượng từng file
4. Chạy cùng 3 query bằng DuckDB, đo thời gian
5. Ghi bảng kết quả vào README

KẾT QUẢ MONG ĐỢI

Parquet-zstd nhỏ nhất, query nhanh nhất

JSON lớn nhất

CÂU HỎI PHỎNG VẤN #08

Parquet khác Avro, khi nào dùng cái nào?

Column cho phân tích, row cho từng bản ghi truyền qua Kafka.

VIDEO TIẾP THEO

09

Data Lake trên S3: tổ chức bronze / silver / gold chuẩn

Thứ Ba 1/12 · 19:00

Đăng ký